NexusAir: A Local-First AI-Powered Knowledge Operating System

Technical Product Research Document — Version 1.0

Jeevananthan S
Department of Computer Science and Engineering
Jain University, Kochi
February 2026 – March 2026 – April 2026

Privacy-First | 100% Local | On-Device LLM | Real-Time Graph | 100% Open-Source | v1.0 Target: Apr 12, 2025

***Abstract*—NexusAir is a *local-first knowledge operating system* that integrates three capabilities previously unavailable in a single privacy-preserving application: block-based document editing, real-time knowledge graph construction, and on-device large language model (LLM) reasoning. Unlike cloud-based tools (Notion, Roam Research) that require transmitting user data to remote servers, or static local editors (Obsidian) that lack AI capabilities, NexusAir executes all computation—including LLM inference, vector similarity search, and graph traversal—on the user's hardware with zero external network calls during runtime. The system employs a Python sidecar architecture (FastAPI + NetworkX + LanceDB + llama-cpp-python) communicating with an Electron/React desktop frontend. A dual-path retrieval mechanism combining graph BFS traversal and vector ANN search, fused via Reciprocal Rank Fusion (RRF), grounds all AI responses in the user's personal knowledge corpus. This document presents the full system design, technology stack, feature specifications, and a ten-week development timeline, four weeks of which are complete as of March 2025.**

***Index Terms*—Knowledge Management, Local-First Software, Large Language Models, Knowledge Graphs, Vector Databases, RAG, Privacy-Preserving AI**

1. Introduction

The proliferation of AI-powered productivity tools has introduced a fundamental tension: the most intelligent tools are cloud-dependent, and the most private tools are static. Knowledge workers managing sensitive information—researchers, consultants, engineers—are forced to choose between capability and confidentiality.

Existing tools address isolated dimensions of the problem. Block-based editors such as Obsidian and Logseq provide flexible local note-taking but lack semantic intelligence. Cloud platforms such as Notion offer AI assistance but require centralizing data on third-party servers. RAG pipeline frameworks such as LangChain enable semantic search but depend on external LLM APIs and significant DevOps overhead.

NexusAir resolves this trilemma through a *unified local-first architecture* that combines all three capabilities in a single desktop application requiring no cloud dependency.

1.1. Research Gap

No existing tool simultaneously satisfies all four criteria: (1) block-based document editing, (2) real-time bidirectional knowledge graph construction, (3) on-device LLM reasoning, and (4) complete data locality. Table 1 documents this gap.

TABLE 1: Competitive capability comparison

System	Edit	Graph	AI	Local
Obsidian	✓	✓	Plugin	✓
Logseq	✓	✓	✗	✓
Notion	✓	✗	✓	✗
Roam	✓	✓	Limited	✗
LangChain	✗	✗	✓	Partial
Neo4j	✗	✓	✗	Partial
NexusAir	✓	✓	✓	✓

1.2. Contributions

This paper makes the following contributions:

1) A unified local-first architecture integrating block editing, graph intelligence, and LLM reasoning.
2) A dual-path retrieval system (graph BFS + vector ANN) fused via RRF for grounded AI responses.

3) A fully open-source implementation stack requiring no paid services or external APIs.
4) A structured ten-week development plan with four completed milestones.

2. System Architecture

NexusAir is a three-process application running entirely on the user's machine. Figure 1 illustrates the layered architecture.

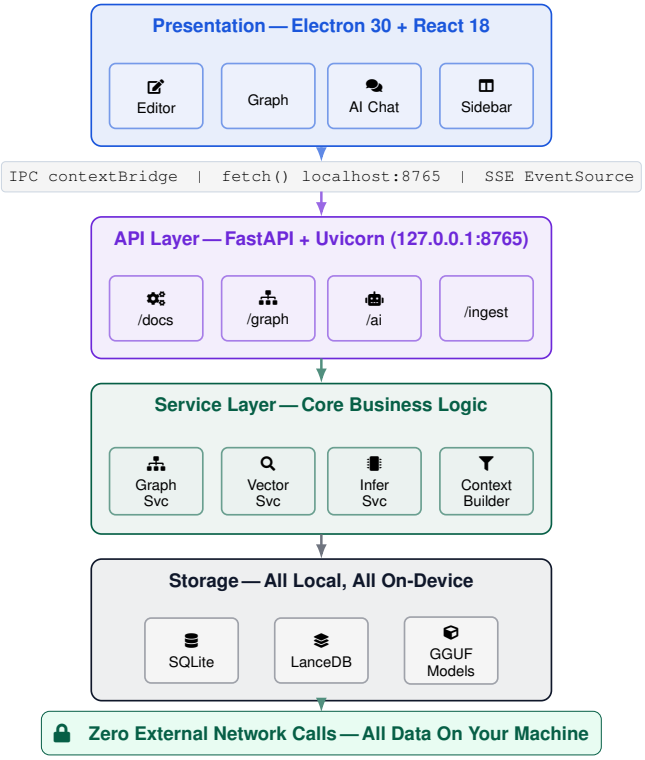

Figure 1: Four-layer architecture. All IPC occurs over localhost; no external network access at runtime.

2.1. Process Model

Three OS-level processes run concurrently:

Electron Main—manages window lifecycle, spawns the Python sidecar via `child_process.spawn()`, establishes the IPC bridge via `contextBridge`, and terminates the sidecar cleanly on quit.

Electron Renderer—the React application inside Chromium's sandbox. All backend calls use `fetch()` to `localhost:8765`; LLM streaming uses `EventSource` (SSE).

Python Sidecar (FastAPI)—bound exclusively to `127.0.0.1:8765`. Initializes `GraphService` (NetworkX), `VectorService` (LanceDB), and `InferenceService` (llama-cpp-python) at startup.

2.2. AI Query Pipeline

Figure 2 shows the end-to-end flow from query to streamed response.

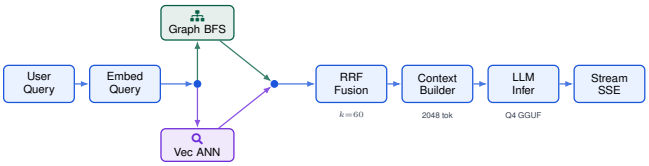

Figure 2: AI query pipeline: parallel dual-path retrieval fused by RRF before context assembly and LLM streaming.

3. Technology Stack

All components are free and open-source. NexusAir requires no paid subscription, API key, or cloud service.

3.1. Frontend — Electron + React (Node.js 20)

TABLE 2: Frontend JavaScript dependencies

Package	Ver.	Lic.	Role
electron	30.0.1	MIT	Desktop shell
react	18.3.1	MIT	UI framework
@codemirror/view	6.26.3	MIT	Editor engine
@cm/lang-markdc	6.2.4	MIT	MD grammar
react-force-graph-2d	1.25.4	MIT	Graph canvas
zustand	4.5.2	MIT	State mgmt
katex	0.16.10	MIT	Math render
tailwindcss	3.4.4	MIT	CSS utility
vite	5.3.1	MIT	Build tool
electron-builde	24.13.3	MIT	Packaging

3.2. Backend — Python 3.11+ Core Engine

TABLE 3: Backend Python dependencies

Package	Ver.	Lic.	Role
API & Server			
fastapi	0.111.0	MIT	REST API
uvicorn	0.30.1	BSD-3	ASGI + SSE
pydantic	2.7.4	MIT	Validation
Graph Engine			
networkx	3.3	BSD-3	DiGraph
markdown-it-py	3.0.0	MIT	AST parser
mdit-py-plugin.	0.4.1	MIT	Wikilinks
watchdog	4.0.1	Apache	FS watch
Vector Layer			
lancedb	0.8.2	Apache	ANN DB
sentence-transformers	3.0.1	Apache	Embeddings
torch (CPU)	2.3.1	BSD-3	ML backend
pyarrow	16.1.0	Apache	Columnar
LLM Inference			
llama-cpp-pyth	0.2.77	MIT	GGUF infer
huggingface-hub	0.23.4	Apache	Model DL
Storage & Ingestion			
sqlalchemy	2.0.31	MIT	Async ORM
aiosqlite	0.20.0	MIT	SQLite drv
pymupdf	1.24.5	AGPL	PDF extract
alembic	1.13.1	MIT	Migrations

3.3. Supported Local LLM Models

Both models are downloaded from HuggingFace Hub—no API key required.

TABLE 4: On-device GGUF models (no API key required)

Model	Disk	RAM	Q	Role
Llama-3-8B-Instruct	4.7 GB	8 GB	Q4_K_M	Default
Phi-3-mini-4k	2.2 GB	5 GB	Q4_K_M	Low-RAM

4. Feature Specifications

NexusAir is built around five core feature pillars. Table 5 gives an overview; the subsections specify each in detail.

TABLE 5: Five core feature pillars

ID	Feature	Library	Status
F1	Block Editor	CodeMirror 6	Done
F2	Knowl. Graph	NetworkX 3.3	Done
F3	Local AI	llama-cpp-python	Active
F4	Dual Retrieval	LanceDB+NX	Wk 6
F5	Doc Ingest	PyMuPDF + ST	Wk 7

4.1. F1 — Block-Based Editor

Built on CodeMirror 6, the editor provides block-level granularity over markdown content.

- **Blocks:** Paragraph, H1–H6, fenced code, LaTeX math, blockquote, embed
- **Wikilinks:** `[[Note Title]]` detected in real-time; resolved links underlined, unresolved dashed
- **Tags:** `#tagname` auto-categorizes documents in sidebar
- **Auto-save:** 500 ms debounced; WAL crash recovery on relaunch
- **Metadata:** YAML frontmatter parsed into properties panel

Acceptance: User creates, links, and navigates 5 documents keyboard-only; auto-save fires within 600 ms; crash recovery restores last state.

4.2. F2 — Knowledge Graph Engine

The graph is a live `networkx.DiGraph` rebuilt from SQLite on startup. Every document is a node; every `[[wikilink]]` is a directed, weighted edge.

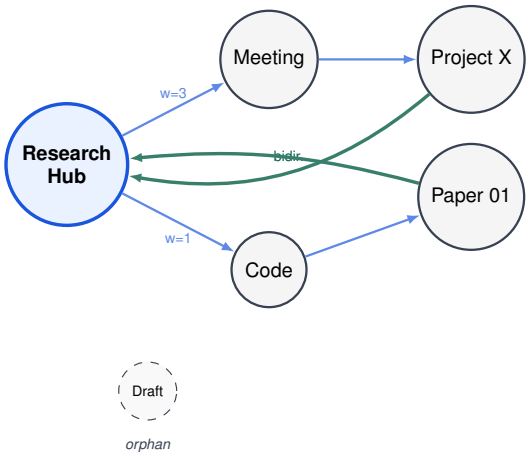

Figure 3: Sample knowledge graph: hub node (high in-degree), weighted/bidirectional edges, orphan node.

Graph updates are *incremental*: on every auto-save, new links are diffed against stored links and only changed edges are modified—no full rebuild needed.

4.3. F3 — Local AI Reasoning

NexusAir uses `llama-cpp-python` to run quantized GGUF models locally. The AI is *grounded-only*—it responds exclusively from retrieved corpus content.

The LLM is always instructed: *"Answer ONLY using the context from the user's knowledge base. If the answer is not in the context, state that explicitly. Cite document titles when referencing information."*

Responses are delivered via Server-Sent Events from `GET /api/ai/stream` enabling token-by-token streaming with a "Stop generation" control.

4.4. F4 — Dual-Path Retrieval & RRF

Two complementary retrieval paths run in parallel:

Graph Path: BFS traversal from query-relevant nodes up to N hops (N=2 default), returning structurally connected documents ordered by hop distance and edge weight.

Vector Path: Query embedded via Sentence Transformers (`all-MiniLM-L6-v2`, 384-dim) and compared via ANN cosine similarity in LanceDB (top-K, default K=20).

Results are fused via Reciprocal Rank Fusion [2]:

$$\mathrm{RRF}(d) = \sum_{r \in \{G,V\}} \frac{1}{k + \mathrm{rank}_r(d)}, \quad k = 60 \qquad (1)$$

Top-N fused results (N=8) are assembled into a 2,048-token context window and passed to the LLM.

TABLE 6: Retrieval path comparison

Property	Graph Path	Vector Path
Algorithm	NetworkX BFS	LanceDB IVF-PQ
Strength	Structural links	Semantic match
Retrieves	Linked neighbors	Related content
Weakness	Misses unlinked	Misses structure

4.5. F5 — Document Ingestion

NexusAir supports Markdown (`.md`) and PDF (`.pdf`) import in v1.0.

TABLE 7: Ingestion capabilities by format

Capability	Markdown	PDF
Text extraction	Native	PyMuPDF
Frontmatter	YAML	Filename
Link resolution	✔	✘
Auto-embedding	✔	✔
Dedup (SHA-256)	✔	✔

5. Development Timeline

NexusAir follows a ten-week plan from February 1 to April 12, 2025. Four weeks are complete as of March 2025.

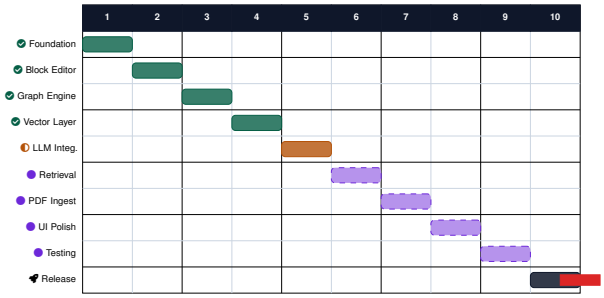

Figure 4: Gantt chart: **Done** (Wks 1–4) · **Active** (Wk 5) · **Planned** (Wks 6–9) · **Release** (Wk 10)

TABLE 8: Completed milestones, Weeks 1–4

Wk	Phase	Key Deliverables
1	Foundation	Monorepo, Electron+Vite, FastAPI sidecar, SQLAlchemy models, Alembic, CI
2	Editor	CodeMirror 6, wikilink ext., `#tag`, auto-save, document CRUD
3	Graph Eng.	NetworkX service, AST parser, incremental updates, graph viz panel
4	Vectors	Sentence Transformers, LanceDB IVF-PQ, async embed, search UI

TABLE 9: Upcoming weeks, definition of done (Weeks 5–10)

Wk	Phase	Definition of Done
5	LLM Integ.	Model loads; first token <3 s on 8 GB RAM
6	Retrieval	Both paths live; RRF fusion; grounding >80%
7	PDF Ingest	10-page PDF queryable within 30 s
8	UI Polish	Onboarding; error states; keyboard shortcuts
9	Testing	All tests pass; P95 latency <200 ms
10	Release	Installers Win/macOS/Linux; v1.0 tag pushed

6. Privacy & Security

> At no point during normal operation does NexusAir transmit user data, queries, embeddings, or AI responses to any external server. Every computation executes on the user's hardware.

NexusAir is architecturally aligned with Kleppmann et al.'s Local-First Software manifesto [1]: offline capability, data ownership, no vendor lock-in, security by default. All seven principles are satisfied by v1.0 design.

TABLE 10: Security controls in v1.0

Control	Implementation	Threat
Network isolat.	Binds `127.0.0.1` only	LAN exposure
IPC sandbox	`contextBridge` safe methods	Renderer rce
XSS prevention	DOMPurify in React	Script inject
Model hash	SHA-256 on GGUF load	Tampered model
Zero telemetry	No analytics shipped	Privacy

7. Performance Requirements

All targets below are hard requirements for v1.0, validated on minimum hardware (4-core CPU, 8 GB RAM, SSD).

TABLE 11: Non-negotiable v1.0 performance targets

Metric	Target	Pctile
App cold start	< 4 s	—
Editor latency	< 16 ms	—
Graph update (save)	< 100 ms	—
Graph render 10k	< 300 ms	—
Retrieval (all)	< 200 ms	P95
LLM first token	< 3 s	—
PDF import 10 pg	< 10 s	—
Corpus graceful	50k blocks	—

AI grounding accuracy target: >80% of responses have all claims traceable to retrieved context on a 50-query eval set.

8. Scope Boundaries

Mobile apps · Cloud sync · Real-time collab · Plugin marketplace · Ungrounded AI mode · Word/Excel import · Paid LLM APIs · Web clipper · Public sharing

9. Conclusion

NexusAir addresses a genuine and previously unsolved gap in personal knowledge management: combining block editing, real-time graph intelligence, and on-device LLM reasoning in a single privacy-preserving desktop application. The dual-path retrieval architecture—graph BFS with vector ANN fused via RRF—forms the technical core and directly determines AI response quality and grounding.

With four of ten development weeks completed and the editor, graph engine, and vector layer fully operational, NexusAir is on schedule for an April 12, 2025 v1.0 release. The entire stack is open-source, the architecture is well-defined, and scope boundaries are explicitly enforced.

NexusAir is not a cloud AI wrapper. It is a genuine local knowledge intelligence system — powerful because it knows your corpus, private because it never leaves your machine.

References

[1] M. Kleppmann, A. Wiggins, P. Van Hardenberg, and M. McGranaghan, "Local-first software: You own your data, in spite of the cloud," in *Proc. ACM SIGPLAN Onward!*, 2019, pp. 154–178.

[2] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning," in *Proc. 32nd ACM SIGIR*, 2009, pp. 758–759.

[3] Meta AI, "Llama 3: Open foundation and fine-tuned chat models," 2024. [Online]. Available: https://llama.meta.com/llama3/

[4] M. Abdin et al., "Phi-3 technical report," *arXiv:2404.14219*, 2024.

[5] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proc. EMNLP*, 2019.

[6] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *NeurIPS*, vol. 33, 2020, pp. 9459–9474.